SYS

시스템 지도와 코드 아틀라스

1,884개 TypeScript 파일을 책임·경계·변경 축으로 읽기

분석 대상
v2.1.88 reconstructed source

고정 리비전
2ca5ddabfed5

독자
아키텍처·플랫폼 학습자

사용 방식
구조 탐색 · 코드 리딩

> 교육·연구용 파생 분석입니다. Anthropic의 공식 소스·공식 아키텍처 문서가 아닙니다.

규모가 아니라 분포를 먼저 본다

라인 수는 난도를 말해 주지 않는다. 어떤 책임이 어디에 몰려 있고, 어떤 파일이 많은 경계를 동시에 연결하는지가 더 중요하다.

1,884	512,664	12,576	1,136
TS/TSX 파일	**물리 라인**	**내부 import edge**	**최대 SCC**
1,332 TS + 552 TSX	고정 리비전 wc 기준	정적 상대/alias import	강한 순환 연결 요소

증거 등급

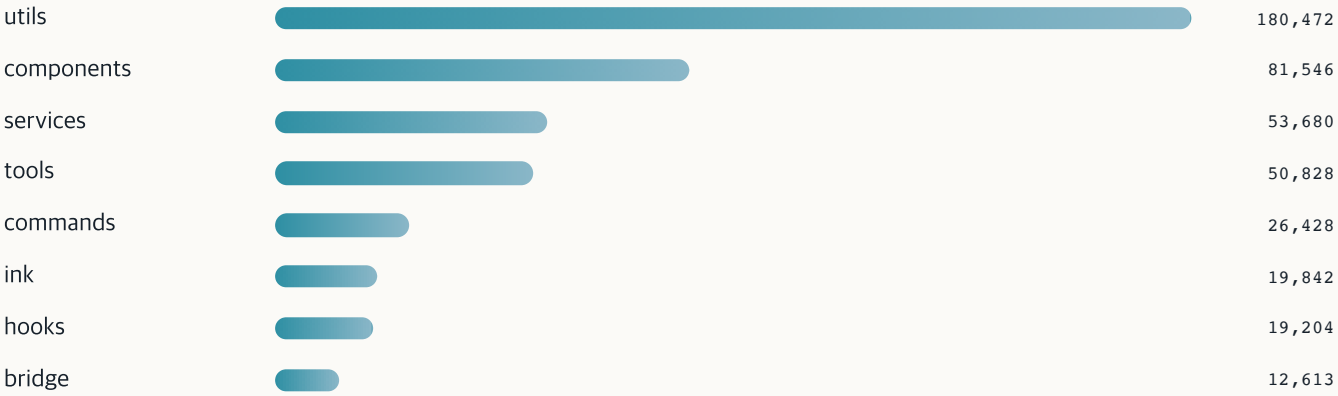
코드 위치와 분포 수치는 고정 커밋의 정적 분석 결과다. 제품 의도·운영 동작·현재 버전에 대한 설명은 별도 검증 없이는 추정으로 취급한다. 저장소 자체가 “decompiled source”라고 밝히며 완전 재빌드 불가와 누락 모듈을 명시한다.

SCC 해석

거대 strongly connected component는 “모든 코드가 나쁘다”는 판정이 아니다. barrel/export, UI와 runtime의 상호 참조, 복원 과정의 평탄화가 함께 만든 변경 위험 신호다.

상위 디렉터리 분포

utils가 코드의 35% 이상을 차지하고, components·tools·services가 다음 세 축을 이룬다.



분포는 “utils = 잡동사니”라고 단정하기보다, 파일시스템·설정·권한·plugin·shell·message 변환 등 횡단 관심사가 어디로 모였는지 조사하라는 신호다.

근거 위치 src/ tree · metrics/source-metrics.json

하나의 제품, 여러 실행 표면

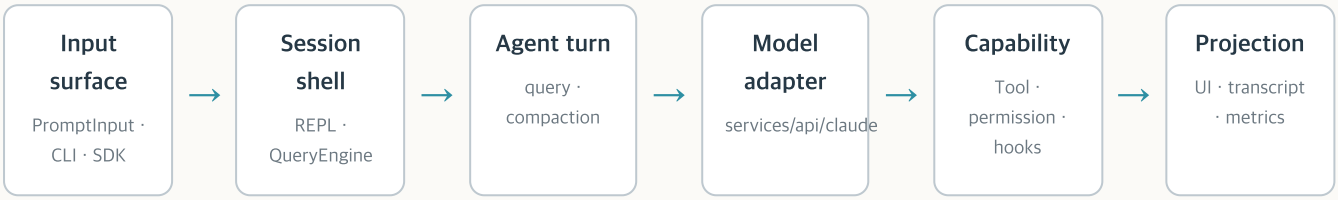
interactive terminal만 보면 UI 상태를 core state로 착각한다. REPL, headless print, SDK, bridge는 같은 능력을 다른 transport와 lifecycle로 감싼다.

표면	입력/출력	소유 관심사	anchor
Interactive REPL	키 입력·Ink frame	prompt UX, live state, permission dialog	src/screens/REPL.tsx
Print/headless	stdin/args · NDJSON/text	비대화형 실행, structured IO	src/cli/print.ts
Agent SDK	typed message/control stream	host integration, schema surface	src/entrypoints/sdk/coreSchemas.ts
QueryEngine	programmatic query lifecycle	headless query orchestration	src/QueryEngine.ts
Bridge/remote	websocket/HTTP/process	session relay, auth, capacity	src/bridge/bridgeMain.ts

> **경계 테스트**
>
> 어떤 상태가 REPL 없이도 필요하면 core/application 상태일 가능성이 높다. spinner나 selection overlay처럼 출력 매체에만 필요한 상태는 adapter/UI 상태다.

사용자 사건이 지나가는 여섯 구역

입력 한 번은 UI 이벤트, 메시지 변환, 에이전트 턴, 모델 스트림, tool capability, persistence/telemetry를 횡단한다.



핵심은 선형 호출이 아니다. 모델 스트림이 UI projection과 tool executor로 분기되고, tool result가 다시 다음 model request의 입력으로 합류한다.

근거 위치 src/query.ts · src/services/api/claude.ts · src/services/tools/toolOrchestration.ts

디렉터리보다 변경 이유로 다시 묶기

복원 소스는 전형적인 계층형 모노레포가 아니다. 학습 시에는 파일 위치보다 “무엇이 바뀔 때 함께 바뀌는가”로 가상 모듈을 만든다.

변경 축	함께 읽을 영역	독립적으로 유지할 계약
새 모델/API 기능	services/api, model utils, betas, message types	request params · stream event · fallback
새 도구	Tool.ts, tools/, toolExecution, UI renderer	schema · permission · result · progress
새 permission policy	types/permissions, permissions, hooks, dialogs	allow/deny/ask · provenance
새 TUI interaction	PromptInput, ink/hooks, keybindings, REPL	key event · focus · submit callback
새 extension	MCP, plugins, skills, commands	discovery · trust · lifecycle
긴 세션 최적화	compact, attachments, storage, tokens	context projection · budget · resume

fan-out이 큰 파일은 조합 지점이다

정적 import fan-out은 책임 수의 직접 측정이 아니지만, 코드를 읽을 순서와 변경 위험을 알려 주는 좋은 탐색 지표다.

파일	내부 fan-out	해석
src/screens/REPL.tsx	222	interactive shell의 조합 중심. UI·session·query·remote를 연결한다.
src/main.tsx	151	부트스트랩과 top-level wiring.
src/cli/print.ts	125	headless surface의 조합 중심.
src/components/PromptInput/PromptInput.tsx	116	입력 UX의 복잡성이 모인 hotspot.
src/commands.ts	85	slash command registry/facade.
src/utils/attachments.ts	83	컨텍스트 enrichment fan-in/out.
src/services/api/claude.ts	60	모델 provider adapter와 stream 변환.

> **리팩터링 오해**
>
> 큰 파일을 자동 분할한다고 응집도가 높아지지 않는다. 먼저 상태 소유권·lifecycle·public contract가 갈라지는지 확인한다.

많이 참조되는 파일은 언어와 기반 시설이다

fan-in이 큰 파일은 변경 비용이 높지만, 동시에 시스템의 ubiquitous language를 가장 빨리 배울 수 있는 지점이다.

파일	fan-in	학습 포인트
src/ink.ts	442	Ink surface/barrel, UI primitive vocabulary
src/utils/debug.ts	369	debugging and diagnostic conventions
src/services/analytics/index.ts	266	event vocabulary and privacy constraints
src/bootstrap/state.ts	249	process/session state concentration
src/utils/errors.ts	248	error taxonomy
src/Tool.ts	218	capability contract
src/state/AppState.tsx	150	interactive global state shape

학습자는 이 파일을 “처음부터 끝까지” 읽기보다, 소비자가 반복해서 요구하는 타입·함수만 인덱스로 만든다.

거대 순환 연결 요소를 어떻게 읽을까

정적 상대 import 그래프에서 8개 SCC가 보이며, 가장 큰 SCC는 1,136파일이다. 이는 변경 전파와 격리 난도를 보여 주는 관찰이지 품질 점수는 아니다.

원인 후보

barrel과 facade

편리한 re-export가 내부 모듈로 다시 들어오면 cycle을 만든다.

원인 후보

UI/runtime 교차

Tool 계약이 React node를 포함하고 runtime이 UI 표현을 참조한다.

원인 후보

global state

AppState·bootstrap singleton·settings가 횡단 관심사를 묶는다.

원인 후보

복원 평탄화

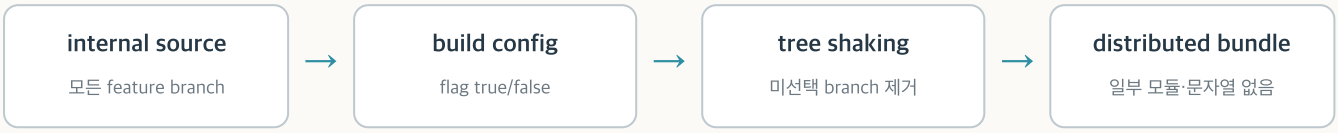
원 모노레포 경계가 단일 src로 복원되며 package boundary가 사라졌을 수 있다.

분석 실습

SCC를 즉시 제거하려 하지 말고, 한 tool vertical slice에서 type-only edge, UI edge, runtime edge를 색으로 구분한다. 잘라야 할 seam이 보일 때만 refactor 가설을 세운다.

Bun feature gate가 소스의 의미를 바꾼다

이 저장소에서 `feature("FLAG")`는 단순 런타임 if가 아니라 compile-time dead-code elimination의 표지로 설명된다.



935	196	419	108
feature() 호출	**bun:bundle import 파일**	**dynamic import**	**누락 모듈 주장**
AST call count	정적 import count	AST call count	저장소 문서 등급 B

중요한 결론

소스 트리에 있는 경로를 보고 출시 제품에 활성이라고 결론 내릴 수 없다. 반대로 missing file을 보고 기능이 존재하지 않는다고도 단정할 수 없다.

근거 위치 QUICKSTART.md · src/query.ts

상태는 하나가 아니라 수명별로 나뉜다

대형 interactive agent는 상태를 수명과 권위로 분리하지 않으면 재개·취소·background task에서 오류가 난다.

수명	대표 상태	소유 위치	실패 예
render frame	focus, overlay, spinner	React hook / Ink	stale closure, flicker
user turn	messages, tool results, abort signal	query State / ToolUseContext	중복 tool result
session	history, permission context, app state	REPL/AppState/storage	resume drift
process	settings cache, task registry, analytics	bootstrap/services singleton	cross-session leak
remote/provider	MCP connection, bridge token, API request	adapter/client	stale auth, partial delivery

근거 위치 src/query.ts · src/Tool.ts · src/state/AppState.tsx

메시지는 UI 문자열이 아니라 프로토콜 상태다

assistant, user, attachment, progress, system, tombstone 등은 렌더링과 API 정규화, 저장 정책이 다르다.

학습용 대수 모델 — 원문 복사 아님

```
type Message =
  | UserMessage
  | AssistantMessage
  | AttachmentMessage
  | ProgressMessage
  | SystemMessage
  | TombstoneMessage

switch (message.type) {
  case "assistant": projectAssistant(message)
  case "attachment": enrichContext(message)
  case "tombstone": removeOrphanProjection(message)
  default: assertExhaustive(message)
}
```

이 모델을 사용하면 “화면에 보인다”, “다음 API 요청에 포함된다”, “세션에 저장된다”를 별도 속성으로 질문할 수 있다.

근거 위치 누락된 message 모듈의 import 지점 · src/utils/messages.ts

> **복원본의 공백**
>
> `src/types/message.ts`는 분석 리비전에 존재하지 않는다. 위 대수는 import/use site에서 재구성한 학습 모델이지 원문 타입 정의가 아니다.

command, skill, plugin, MCP, tool은 같은 것이 아니다

확장 계층마다 발견 방식과 실행 권한이 다르다. 이름이 비슷해도 서로 대체하지 않는다.

개념	주 역할	실행 주체/경계	위험 질문
Command	사용자 입력을 local action 또는 prompt로 해석	REPL/command processor	즉시 실행인가 모델 입력인가?
Skill	instruction bundle·도메인 절차 제공	SkillTool/context injection	instruction이 authority처럼 취급되는가?
Plugin	commands, agents, hooks, MCP 등을 묶어 배포	plugin loader/manager	설치 출처와 업데이트 신뢰는?
MCP	외부 server가 tool/resource/prompt 노출	MCP client/transport	서버·annotation·result를 신뢰하는가?
Tool	모델이 호출 가능한 typed capability	tool runtime	schema·permission·effect가 닫혔는가?

근거 위치 src/commands.ts · src/tools/SkillTool/SkillTool.ts · src/utils/plugins/pluginLoader.ts · src/services/mcp/client.ts

모델에 들어가는 것은 대화 기록만이 아니다

메시지, 시스템 프롬프트, user/system context, memory, skill discovery, queued command, file-change attachment가 turn 시점에 조립된다.



아키텍처 의미

컨텍스트는 정적 문자열이 아니라, source·budget·freshness·persistence 규칙을 가진 실행 시점 projection이다.

근거 위치 src/query.ts · src/query.ts · src/utils/attachments.ts

Enter 한 번의 구조적 추적

함수 이름을 외우기보다 책임 전환점을 표시한다.

키 이벤트

Ink input hook이 Enter를 구조화된 key로 전달한다.

UX gate

selection, suggestion, empty input, teammate routing 등 조기 종료 조건을 적용한다.

session submit

history·slash command·remote 여부·abort controller를 준비한다.

query boundary

UI state와 agent turn을 연결하고 stream event를 소비한다.

model/tool loop

compact → model stream → tool execution → follow-up.

projection

message state, Static history, transcript, telemetry가 각자 갱신된다.

근거 위치 src/components/PromptInput/PromptInput.tsx · src/screens/REPL.tsx · src/query.ts

파일 하나를 읽을 때 기록할 여덟 문장

호출 그래프보다 책임과 불변식을 기록하면 다른 프로젝트에도 지식이 전이된다.

- ☐ 이 파일이 소유하는 상태/결정은 무엇인가?
- ☐ 입력과 출력의 typed contract는 무엇인가?
- ☐ 성공·거부·취소·오류는 어떤 형태로 드러나는가?
- ☐ 외부 효과 직전 마지막 검증 지점은 어디인가?
- ☐ 상태의 수명은 frame/turn/session/process 중 무엇인가?
- ☐ 이 파일이 UI·provider·filesystem에 묶이는 지점은 어디인가?
- ☐ feature gate나 runtime setting이 의미를 바꾸는가?
- ☐ 테스트할 때 관찰 가능한 결과는 무엇인가?

실습 · vertical slice

`FileReadTool` 하나를 골라 schema → permission → execution → result → render를 한 장에 그린다.

완료 증거: 파일/line ref가 붙은 5단계 다이어그램

구조를 읽고 남겨야 할 결론

이 저장소의 가장 중요한 구조적 특징은 “작은 agent loop 주위에 거대한 product harness가 붙어 있다”는 점이다.

핵심

작은 제어 루프

model stream과 tool result를 번갈아 잇는 loop는 비교적 명확하다.

제품성

큰 harness

UI, permissions, persistence, remote, plugins, observability가 실사용 품질을 만든다.

위험

강한 결합

REPL·global state·utils 허브는 변경 영향과 읽기 난도를 높인다.

학습법

수직 절단

한 사용자 사건이나 tool을 끝까지 추적하는 것이 디렉터리 순회보다 낫다.

다음 권

이 구조를 실제로 가능하게 한 언어 장치—async generator, union narrowing, generic Tool contract, cancellation—를 02권에서 해부한다.